

Lecture #2.

Pinocchio Examples

Kyung Hee University
Team member: Eunae Kang, Chanjin Park
Prof. Sanghyun Kim



01

Loading a model

Loading a model | Prerequisites

You need to export robots' model directory before execution.

Clone model repository

➤ At pinocchio dir:

```
cd models
git clone https://github.com/Gepetto/example-robot-data.git
```

Exporting directory path

➤ On current terminal:

```
export EXAMPLE_ROBOT_DATA_MODEL_DIR=(path_to_pinocchio)/models/example-robot-data/robots
```

➤ Add into .bashrc (optional):

```
echo 'export
EXAMPLE_ROBOT_DATA_MODEL_DIR=(path_to_pinocchio)/models/example-robot-
data/robots' >> ~/.bashrc
```

02

**Computes analytical derivatives
of rigid body dynamics algorithms**

Computes analytical derivatives of rigid body dynamics algorithms

Analytical derivatives (Remind)

- Pinocchio provides efficient implementations of analytical derivatives of these algorithms. These derivatives are particularly important for whole-body trajectory optimization, and more generally for numerical optimal control.

Learning Objectives

By the end of this lecture, you should be able to:

- Compute analytical derivatives of joint velocities and accelerations using Pinocchio.
- Understand how a joint's velocity and acceleration change.
- Access derivatives in both world and local coordinate frames.

03

Displaying a model

Displaying a model | Outline

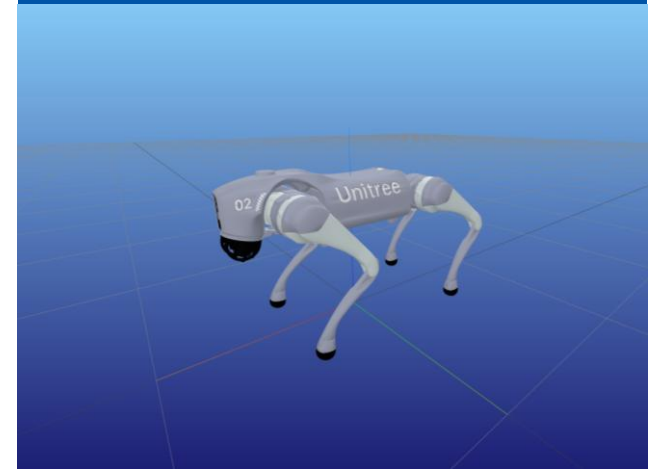
Learning Objectives

Displaying a model is the most intuitive way to understand how a robot is built and how it moves.

By the end of this lecture, you should be able to:

- Load a robot model from a URDF file using Pinocchio
- Understand the structure of the code for displaying a model
- Visualize the robot using Meshcat

Example of displaying a model



RViz



Gazebo



Unity

Displaying a model | Prerequisites

For the following examples, you should have [gepetto-gui](#) or [MeshCat](#) installed.

Installation of 3D viewers

▶ Installing meshcat:

```
pip install meshcat
```

▶ Installing gepetto-viewer:

```
conda install gepetto-viewer-corba -c conda-forge
```

▶ Installing panda3d_viewer:

```
pip install panda3d_viewer
```


Displaying a model | Prerequisites

You need to export robots' model directory before execution.

Clone model repository

➤ At pinocchio dir:

```
cd models
git clone https://github.com/Gepetto/example-robot-data.git
```

Exporting directory path

➤ On current terminal:

```
export EXAMPLE_ROBOT_DATA_MODEL_DIR=(path_to_pinocchio)/models/example-robot-data/robots
```

➤ Add into .bashrc (optional):

```
echo 'export
EXAMPLE_ROBOT_DATA_MODEL_DIR=(path_to_pinocchio)/models/example-robot-
data/robots' >> ~/.bashrc
```

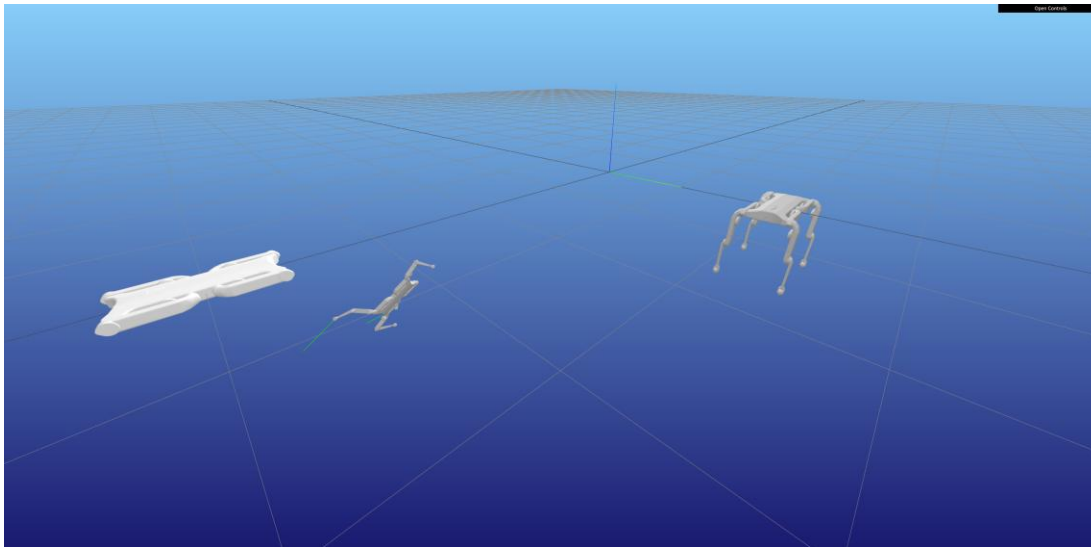
Displaying a model | Example (1)

- ▶ Loading a robot model using MeshCat:

```
python -i meshcat-viewer.py
```

- ▶ The following output will appear when you run the code:

```
(pin) home@DESKTOP-T4D06E6:~/pinocchio/examples$ python meshcat-viewer.py  
You can open the visualizer by visiting the following URL:  
http://127.0.0.1:7001/static/
```



Displaying a model | [Explanation of the code \(meshcat-viewer.py\)](#)

▶ Loading the URDF model

```
pinocchio_model_dir = Path(__file__).parent.parent / "models"

model_path = pinocchio_model_dir / "example-robot-data/robots"
mesh_dir = pinocchio_model_dir
urdf_filename = "solo.urdf"
urdf_model_path = model_path / "solo_description/robots" / urdf_filename

model, collision_model, visual_model = pin.buildModelsFromUrdf(
    urdf_model_path, mesh_dir, pin.JointModelFreeFlyer()
)
```

buildModelsFromUrdf ▶ loads the robot's URDF and automatically builds its models on defined root joint type

JointModelFreeFlyer ▶ defines a 6-DoF floating base joint, allowing the robot's base to move freely in space.

Displaying a model | [Explanation of the code \(meshcat-viewer.py\)](#)

▶ Starting a new MeshCat server and client

```
try:
    viz = MeshcatVisualizer(model, collision_model, visual_model)
    viz.initViewer(open=True)
except ImportError as err:
    print(
        "Error while initializing the viewer. "
    )
    print(err)
    sys.exit(0)

# Load the robot in the viewer.
viz.loadViewerModel()

# Display a robot configuration.
q0 = pin.neutral(model)
viz.display(q0)
viz.displayVisuals(True)
```

`pin.neutral(model)` ▶ returns the default or neutral configuration of the given robot model

Displaying a model | [Explanation of the code \(meshcat-viewer.py\)](#)

► Creating and displaying the convex object on the scene

```
mesh = visual_model.geometryObjects[0].geometry
mesh.buildConvexRepresentation(True)
convex = mesh.convex

if convex is not None:
    placement = pin.SE3.Identity()
    placement.translation[0] = 2.0
    geometry = pin.GeometryObject("convex", 0, placement, convex)
    geometry.meshColor = np.ones(4)
    geometry.overrideMaterial = True
    geometry.meshMaterial = pin.GeometryPhongMaterial()
    geometry.meshMaterial.meshEmissionColor = np.array([1.0, 0.1, 0.1, 1.0])
    geometry.meshMaterial.meshSpecularColor = np.array([0.1, 1.0, 0.1, 1.0])
    geometry.meshMaterial.meshShininess = 0.8
    visual_model.addGeometryObject(geometry)
    # After modifying the visual_model we must rebuild
    viz.rebuildData()
```

pin.SE3.Identity creates an identity pose in SE(3); a transformation with zero translation and no rotation

pin.GeometryObject defines a single geometric object (name, parent_joint, placement, collision_geometry)

addGeometryObject adds a new object to the robot's visual model so that it can be rendered in the viewer

Displaying a model | [Explanation of the code \(meshcat-viewer.py\)](#)

► Displaying another robot in standing configuration

```
viz2 = MeshcatVisualizer(model, collision_model, visual_model)
viz2.initViewer(viz.viewer)
viz2.loadViewerModel(rootNodeName="pinocchio2")
q = q0.copy()
q[1] = 1.0
viz2.display(q)

# standing config
q1 = np.array([0.0, 0.0, 0.235, 0.0, 0.0, 0.0, 1.0, 0.8, -1.6, 0.8, -1.6, -
0.8, 1.6, -0.8, 1.6])
v0 = np.random.randn(model.nv) * 2????????
data = viz.data
pin.forwardKinematics(model, data, q1, v0)
frame_id = model.getFrameId("HR_FOOT")
viz.display()
viz.drawFrameVelocities(frame_id=frame_id)
model.gravity.linear[:] = 0.0
dt = 0.01
```

model.nv gives the number of velocity degrees of freedom in the robot model

pin.forwardKinematics computes the position, velocity, and acceleration of all joints in the robot based on the given configuration, updating the results inside the 'data'

Displaying a model | [Explanation of the code \(meshcat-viewer.py\)](#)

► Updating pose over time with ABA and integration

```
def sim_loop():
    tau0 = np.zeros(model.nv)
    qs = [q1]
    vs = [v0]
    nsteps = 100
    for i in range(nsteps):
        q = qs[i]
        v = vs[i]
        a1 = pin.aba(model, data, q, v, tau0)
        vnext = v + dt * a1
        qnext = pin.integrate(model, q, dt * vnext)
        qs.append(qnext)
        vs.append(vnext)
        viz.display(qnext)
        viz.drawFrameVelocities(frame_id=frame_id)
    return qs, vs

qs, vs = sim_loop()
```

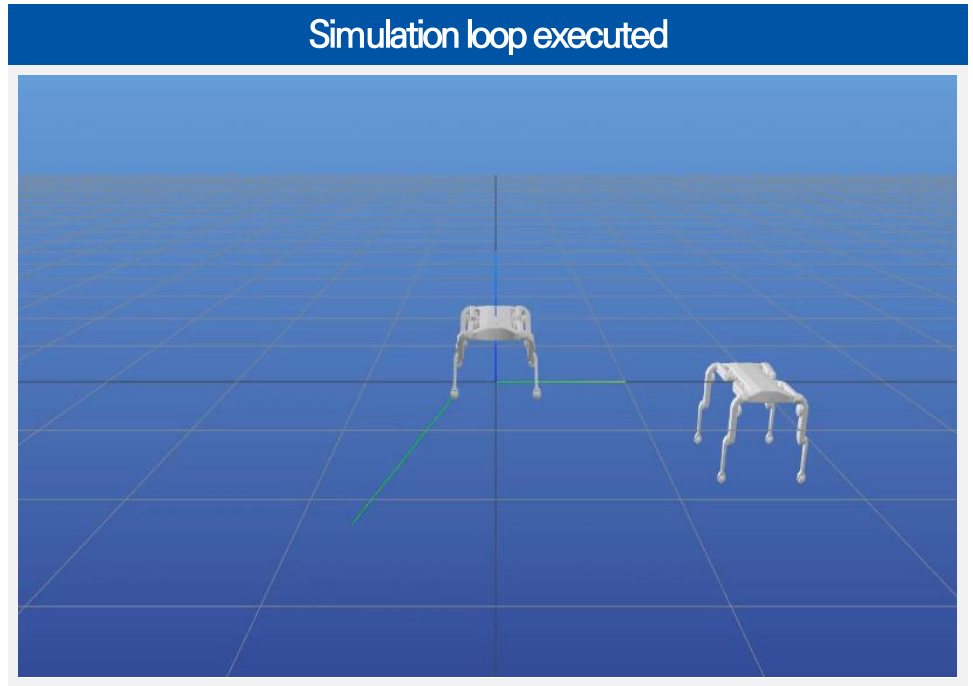
pin.aba ► implements the Articulated-Body Algorithm (ABA) to compute the robot's generalized accelerations

pin.integrate ► updates the robot's configuration by integrating the velocity over time

Displaying a model | Execution of the code (meshcat-viewer.py)

Meshcat viewer

- At the beginning, the robot URDF is displayed in the neutral configuration and standing configuration(user-defined).
- When the code is executed, the simulation starts with random joint velocities, as shown in the video on the right.



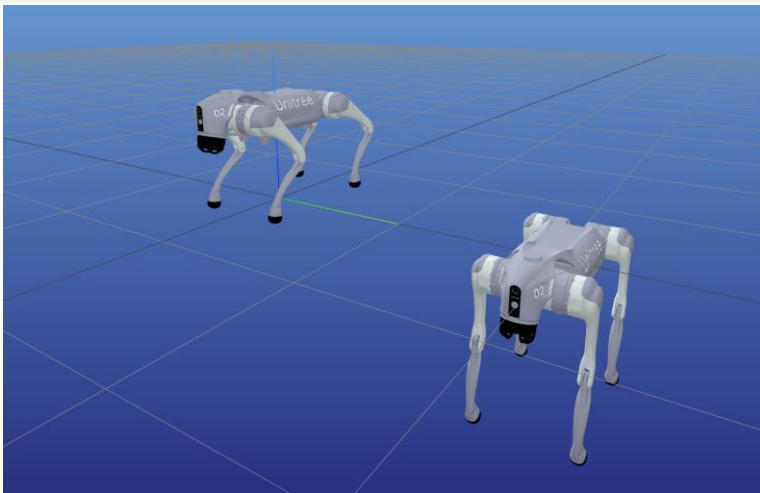
Displaying a model | Execution of the code (meshcat-viewer.py)

Meshcat viewer

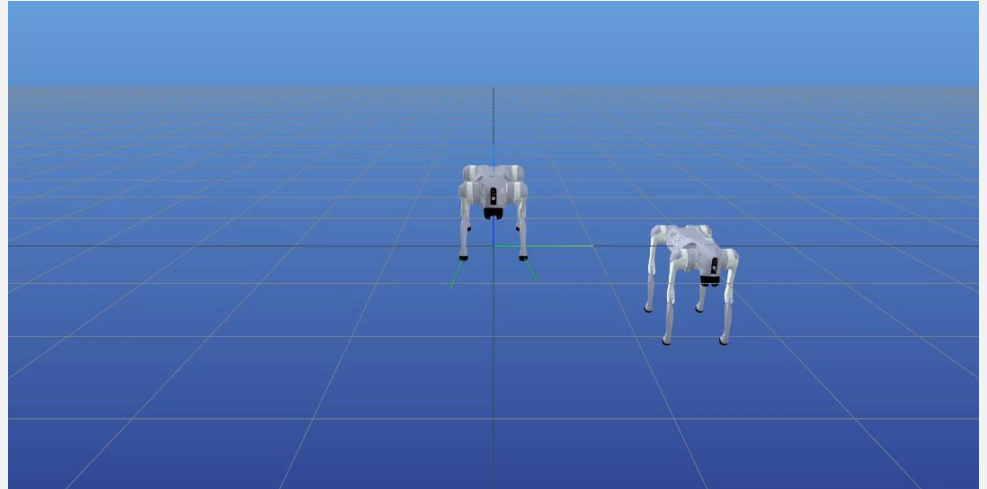
- By setting q to specific configuration, various postures of different robots can be displayed.

```
urdf_filename = "go2.urdf"  
urdf_model_path = model_path / "go2_description/urdf" / urdf_filename  
  
q1 = np.array([0.0, 0.0, 0.32, 0.0, 0.0, 0.0, 1.0, 0.0, 0.8, -1.5, 0.0, 0.8, -  
1.5, 0.0, 1.0, -1.5, 0.0, 1.0, -1.5])  
  
frame_id = model.getFrameId("FR_foot")
```

go2 in neutral and standing configuration



Simulation loop executed

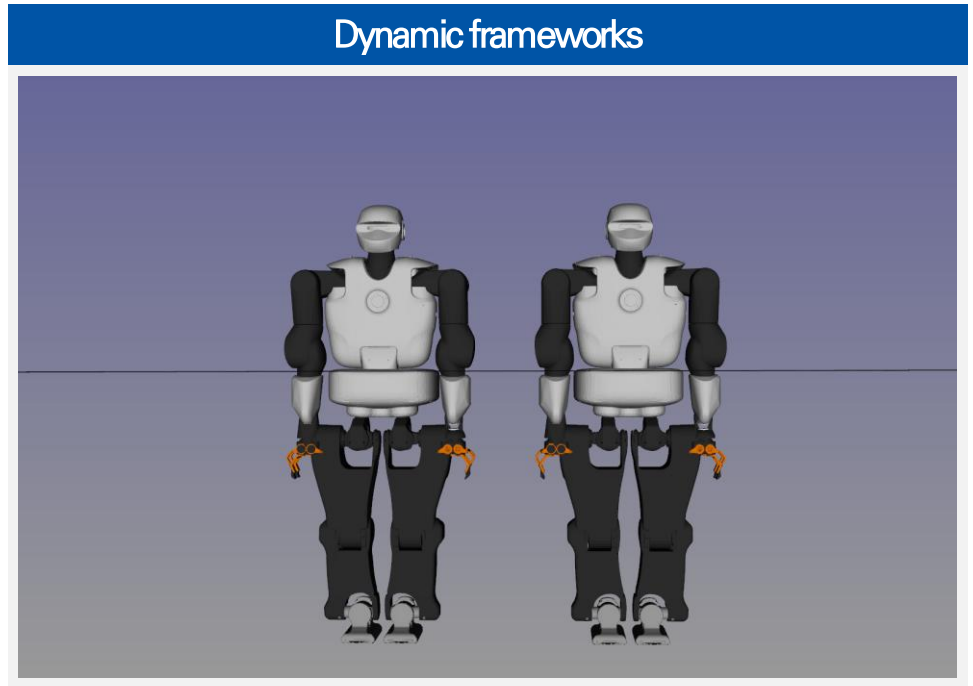


Displaying a model | Execution of the code (gepetto-viewer.py)

Gepetto viewer

- Gepetto-gui is a Graphical Interface for Pinocchio and HPP
- Wrapped inside the Pinocchio library's visualizer as a class called 'GepettoVisualizer'
- Loading a robot model using gepetto-gui:

```
python -i gepetto-viewer.py
```



04

Checking collisions

Checking collisions | Outline

Learning Objectives

By the end of this lecture, you should be able to:

- Configure a collision geometry model and collision pairs from URDF and SRDF in Pinocchio
- Check collisions and distances for a given configuration using `pin.computeCollisions` and `pin.computeDistances`
- Interpret collision results to identify colliding links and apply the pattern to posture validation and point cloud based environment checking



Why collision detection is important

► Motion Planning

- Collision detection is critical for ensuring robots avoid obstacles during motion

► Simulation & Control

- Used in simulation to control with collision-free motion trajectories
- Essential for real-time safe robot operation in unpredictable environments

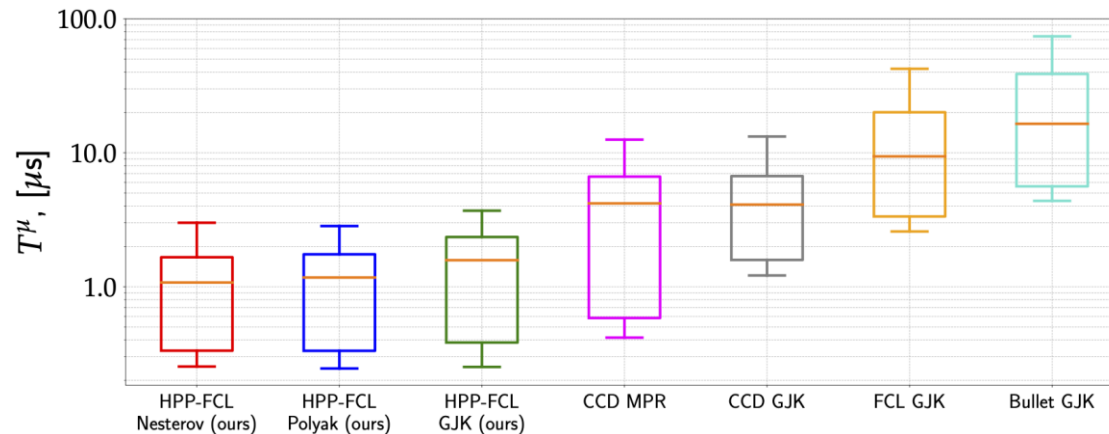
Checking collisions | Collision Library

FCL (Flexible Collision Library)

- ▶ (continuous) Collision detection, Distance computation, Tolerance verification
- ▶ Supports various object shapes: mesh, convex, primitive shapes(e.g. box, sphere, etc.)

HPP-FCL / Coal

- ▶ [FCL](#) was forked in 2015, creating a new project called HPP-FCL and renamed to [Coal](#).
- ▶ New features such as safety margin, distance lower bound, Python bindings
- ▶ Implements the well-establishes GJK algorithm, largely outperform by a large margin (from 5x up to 15x times faster)



	$N_v = 8$ $N_f = 6$	$N_v = 250$ $N_f = 496$	$N_v = 940$ $N_f = 1876$
ProxQP	$5.3 \pm 2.7 \mu s$	$(2 \pm 0.6) \cdot 10^3 \mu s$	$(20 \pm 14) \cdot 10^3 \mu s$
GJK	$0.2 \pm 0.03 \mu s$	$0.8 \pm 0.3 \mu s$	$2.1 \pm 0.5 \mu s$
Ours	$0.2 \pm 0.05 \mu s$	$0.7 \pm 0.2 \mu s$	$1.4 \pm 0.3 \mu s$

How Pinocchio Encapsulates FCL

- ▶ Pinocchio uses FCL for collision detection within its dynamics framework.
- ▶ Pinocchio's `geom_model` and model objects are used for robot collision modeling and physics simulations.

Functions in Pinocchio for Collision Detection

- ▶ `pin.computeCollisions()`: Computes collision information for given configurations.
- ▶ `pin.computeDistances()`: Computes minimum distances for all active pairs and stores the results in `geom_data.distanceResults`
- ▶ `pin.updateGeometryPlacements()`: Updates the geometric placements for collision checking

What does the geometryObjects store?

A list of all geometry objects used for collision/distance checking

```
geom1 = geom_model.geometryObjects[i]
```

- ▶ **geom1.name**

The name string of a geometry object, used to identify which shape or link it belongs to

- ▶ **geom1.parentJoint**

The index of the joint to which this geometry object is attached in the kinematic model

- ▶ **geom1.geometry**

The underlying geometric shape/mesh (e.g., FCL object) used for collision and distance computations.

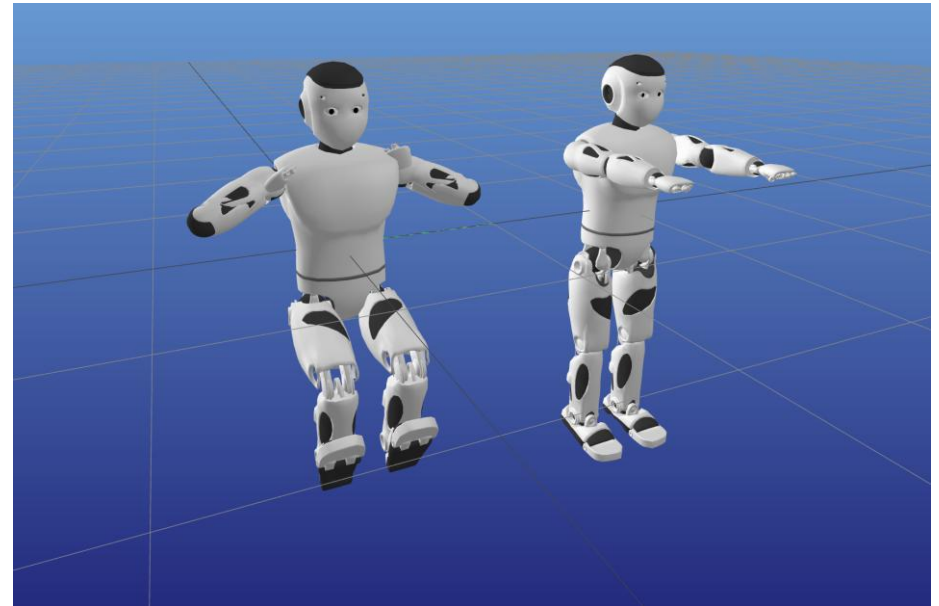
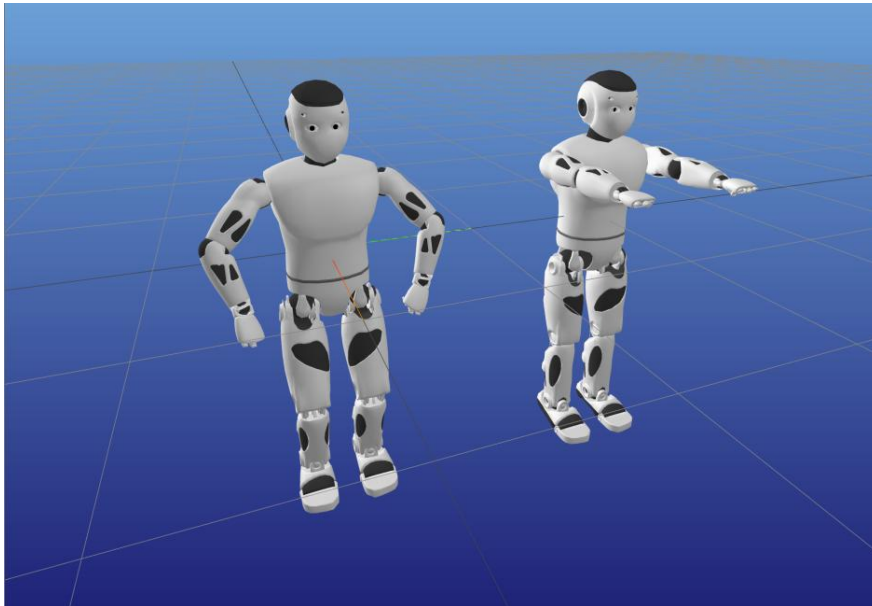
- ▶ **geom1.placement**

The fixed transform (SE3) from the parent joint/frame to the geometry object's local frame

Checking collisions | [Example code \(collisions.py\)](#)

- ▶ Add and remove collision pairs using `geom_model` and SRDF file
- ▶ Load reference configuration (e.g., `half_sitting`) and check if there is any collision
- ▶ Compute all existing collisions with `computeCollisions` and inspect `collisionPairs` / `collisionResults`
- ▶ `export EXAMPLE_ROBOT_DATA_MODEL_DIR` if not added into `~/.bashrc`

```
export EXAMPLE_ROBOT_DATA_MODEL_DIR=(path_to_pinocchio)/models/example-robot-data/robots
```



Checking collisions | [Example code \(collisions.py\)](#)

▶ Loading a model and its collision geometries

```
import os
from pathlib import Path

import pinocchio as pin

model_path = Path(os.environ.get("EXAMPLE_ROBOT_DATA_MODEL_DIR"))
mesh_dir = model_path.parent.parent
urdf_filename = "romeo_small.urdf"
urdf_model_path = model_path / "romeo_description/urdf" / urdf_filename

model = pin.buildModelFromUrdf(urdf_model_path, pin.JointModelFreeFlyer())

geom_model = pin.buildGeomFromUrdf(
    model, urdf_model_path, pin.GeometryType.COLLISION, mesh_dir
)
```

Checking collisions | [Example code \(collisions.py\)](#)

- ▶ Adding and removing collision pairs using `geom_model` and `srdf` file

```
geom_model.addAllCollisionPairs()
print("num collision pairs - initial:", len(geom_model.collisionPairs))

srdf_filename = "romeo.srdf"
srdf_model_path = model_path / "romeo_description/srdf" / srdf_filename

pin.removeCollisionPairs(model, geom_model, srdf_model_path)
print(
    "num collision pairs - after removing useless collision pairs:",
    len(geom_model.collisionPairs),
)
```

- addAllCollisionPairs** ▶ Adds all possible collision pairs between geometry objects to the list `geom_model.collisionPairs`
- removeCollisionPairs** ▶ Removes specified collision pairs from the list, based on configurations defined in the SRDF file to avoid unnecessary checks.

Checking collisions | [Example code \(collisions.py\)](#)

▶ Loading reference configuration and computing all the collisions

```
pin.loadReferenceConfigurations(model, srdf_model_path)

q = model.referenceConfigurations["half_sitting"]

data = model.createData()
geom_data = pin.GeometryData(geom_model)

pin.computeCollisions(model, data, geom_model, geom_data, q, False)
```

pin.loadReferenceConfigurations ▶ Loads predefined reference configurations (e.g., "standing", "half_sitting") from an SRDF file into the model.

referenceConfigurations ▶ Stores a dictionary of reference configurations by name, representing specific robot postures that can be quickly loaded for collision checks or simulation.

pin.computeCollisions ▶ Computes collisions for all active collision pairs at a given configuration and stores the results in `geom_data.collisionResults`

Checking collisions | [Example code \(collisions.py\)](#)

- ▶ Print the status of collision for all collision pairs

```
for k in range(len(geom_model.collisionPairs)):
    cr = geom_data.collisionResults[k]
    cp = geom_model.collisionPairs[k]

    print(
        "collision pair:",
        cp.first,
        ", ",
        cp.second,
        "- collision:",
        "Yes" if cr.isCollision() else "No",
    )
```

Checking collisions | [Example code \(collisions.py\)](#)

- ▶ Print the status of collision for all collision pairs

```
for k in range(len(geom_model.collisionPairs)):
    cr = geom_data.collisionResults[k]
    cp = geom_model.collisionPairs[k]

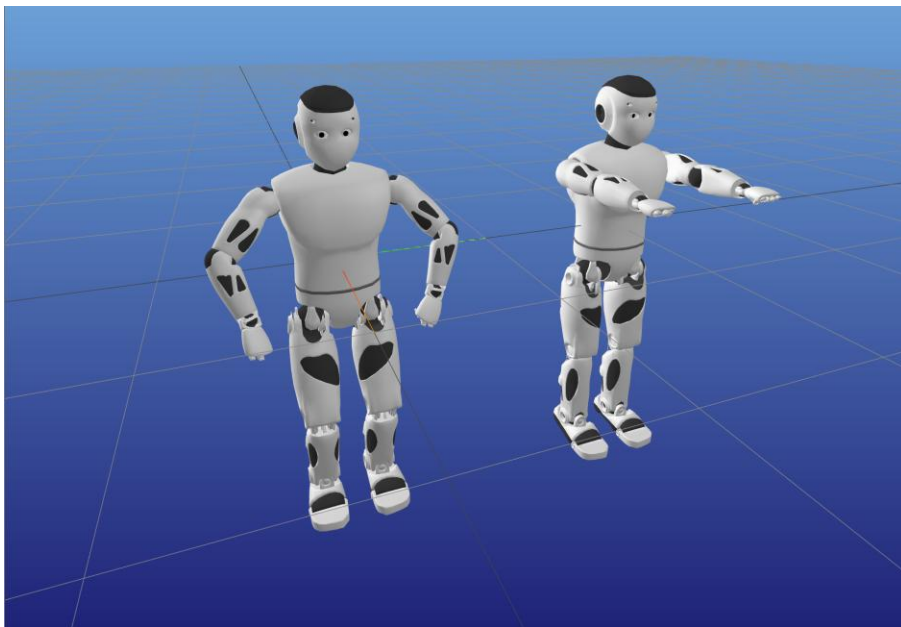
    obj1 = geom_model.geometryObjects[cp.first]
    obj2 = geom_model.geometryObjects[cp.second]

    if cr.isCollision():
        print(
            "collision pair:",
            obj1.name,
            ", ",
            obj2.name
        )
```

Checking collisions | [Example code \(collisions.py\)](#)

► Result with visualization

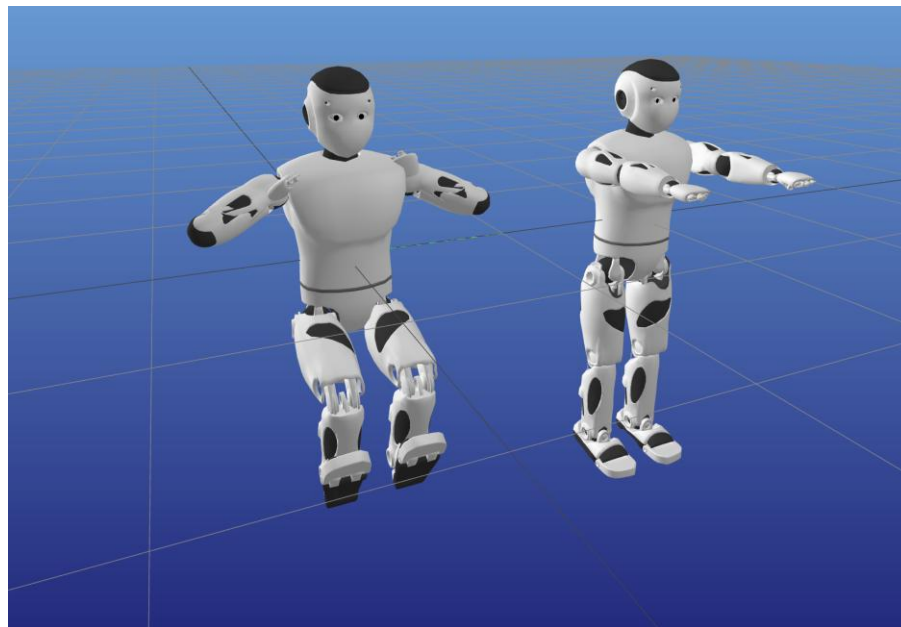
“half_sitting” Configuration



```
q = model.referenceConfigurations["half_sitting"]
```

No collision detected

“half_sitting*2” Configuration



```
q = model.referenceConfigurations["half_sitting*2"]
```

collision pair: torso_0 , l_wrist_0

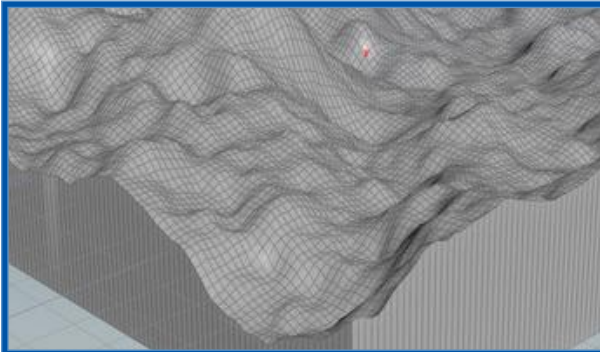
collision pair: torso_0 , r_wrist_0

Checking collisions | [Example code \(collision-with-point-cloud.py\)](#)

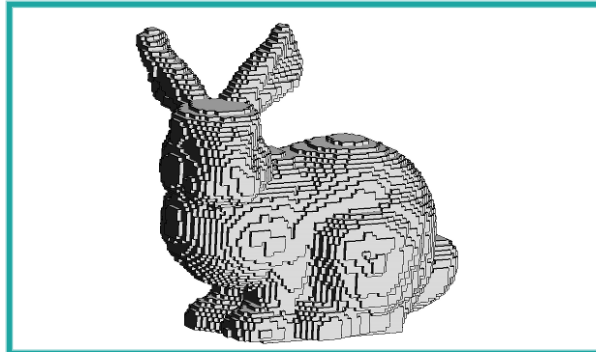
Execution of “collision-with-point-clouds.py”

- ▶ Performs collision detection between the end-effector of a robot and a point cloud
- ▶ Converts point cloud data to a usable collision representation (height field / voxel grid / etc.)
- ▶ Enables integration of sensors that provide information as point clouds.

Types of collision representation



Height field

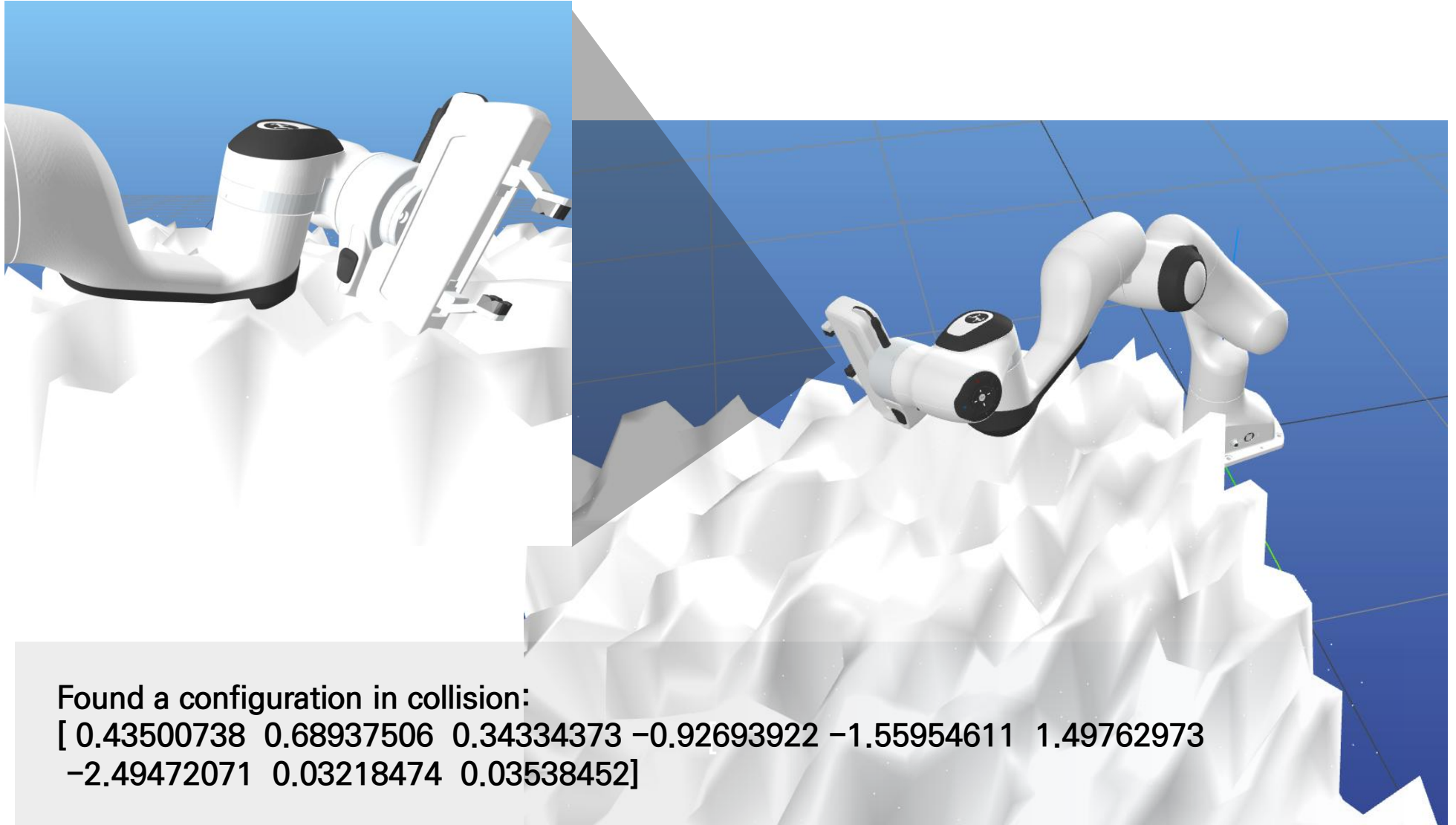


Voxel grid



Convex decomposition

Checking collisions | [Example code \(collision-with-point-cloud.py\)](#)



Checking collisions | Summary

- ▶ Collision detection is fundamental for safe robot operation in real-world environments
- ▶ FCL provides an efficient and flexible library for collision detection, and Pinocchio integrates this into its dynamics engine
- ▶ Practical examples show how to implement collision detection in real scenarios, such as with point cloud data or self-collision checks
- ▶ Future trends include differentiable collision detection for motion optimization and learning-based approaches

Thank You